ARCHITECTURAL & ENGINEERING SPECIFICATION

SentinelQA: The Flaky Testing Nightmare

Autonomous UI Test Generation, Execution & Runtime Self-Healing from Jira Acceptance Criteria

NODE.JS + TYPESCRIPT | REACT + VITE | PLAYWRIGHT AUTOMATION | GEMINI LLM AGENT

1. EXECUTIVE SUMMARY

ATTRIBUTE	DETAILS
Title	**SentinelQA: The Flaky Testing Nightmare**
Domain	AI × Software Testing × DevOps / CI-CD
Core Challenge	Eliminate the bottleneck between "code complete" and "QA tested" by generating, executing, and self-healing UI tests strictly derived from Jira Acceptance Criteria.
Key Value Metric	Zero-to-minimal manual QA script maintenance when frontend element locators, labels, or layout attributes shift.

2. DECONSTRUCTION OF PROBLEM & REQUIREMENTS

A. The Core Problem (The Gap):

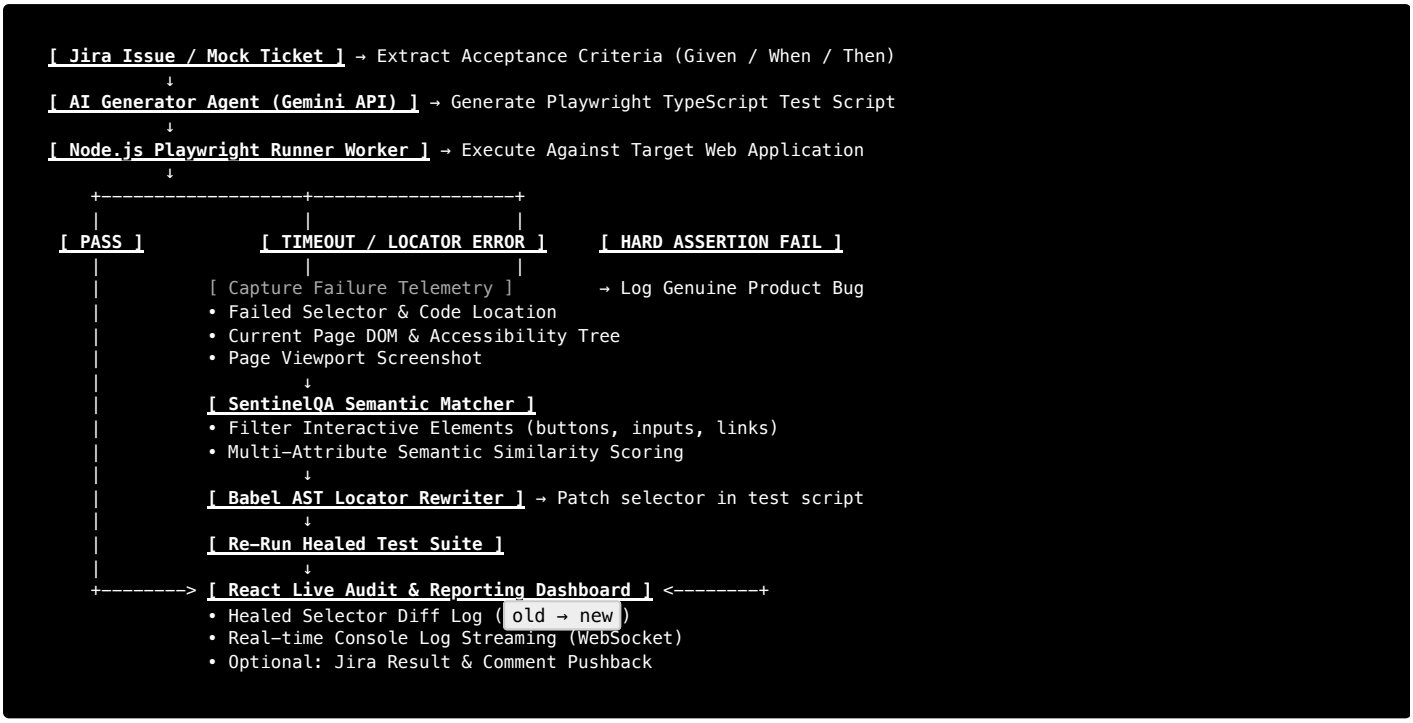
- **Authoring Friction:** Translating English user story acceptance criteria (AC) into reliable end-to-end tests (Playwright) requires significant manual engineering time.
- **Brittle Locators (Test Flakiness):** Minor developer refactors (changing an ID from `#btn-submit` to `#checkout-confirm`, tweaking Tailwind CSS classes, or altering text from "Submit" to "Place Order") break tests even when the feature functions properly.
- **Maintenance Burden:** QA engineers spend 30–50% of their sprints updating broken locators instead of testing new features.

3. MINIMUM WORKING VERSION (MVP) BREAKDOWN

According to Page 20 of PS.pdf, the prototype must deliver all 5 core components:

COMPONENT	PROBLEM STATEMENT REQUIREMENT	NODE.JS + REACT IMPLEMENTATION
1. Ticket Input	Ingest acceptance criteria from Jira (live Jira REST API or mock ticket JSON/UI input).	Node.js Express backend calling Jira Cloud REST API (`/rest/api/3/issue/{key}`) with a React UI dropdown for sample tickets (E-Commerce, Auth, Todo).
2. Test Generation	Automatically synthesize executable UI test scripts from the criteria.	Gemini API generates structured TypeScript/Playwright test suites with assertion steps mapped directly to criteria.
3. Test Execution	Run the tests against a target web app and log pass/fail status.	Node.js child-process runs Playwright test runner in headless mode, streaming execution logs, step timings, and failure screenshots over WebSockets to React.
4. Self-Healing Engine *(Core Differentiator)*	When a test fails due to locator mismatch, automatically locate the updated element, patch the test script, and re-execute.	Intercept locator timeout → extract live DOM & accessibility tree → LLM semantic candidate ranking → Babel AST locator rewrite → automatic re-execution.
5. Audit & Reporting	Visual dashboard detailing tests passed, failed, and healed.	React dashboard rendering step timelines, before/after locator diffs, repair confidence scores, and script change audit logs.

4. DEEP-DIVE: DESIGNING THE SELF-HEALING ENGINE



• DISTINGUISH LOCATOR DRIFT VS TRUE REGRESSIONS

Locator Drift: The element still exists and carries out the intended user action, but its ID, CSS classes, or inner text shifted during refactoring. → **Self-Heal.**

True Bug / Regression: The backend API returns 500, data is not saved, or the element was completely deleted without replacement. → **Report Product Bug.**

• MULTI-ATTRIBUTE CANDIDATE SCORING

Calculate semantic and structural similarity across four key axes:

1. **Tag & ARIA Role (25%):** Confirms candidate matches interaction type (`button`, `link`, `input`).
2. **Text Similarity (35%):** Cosine similarity between expected label and current candidate text.
3. **DOM Hierarchy (20%):** Proximity to sibling fields, forms, and parent containers.
4. **Accessibility Attributes (20%):** Matches `aria-label`, `name`, and placeholders.

• BABEL AST LOCATOR REWRITER (SAFE DETERMINISTIC CODE PATCHING)

Instead of prompting an LLM to regenerate the entire script (which risks syntax errors, hallucinated test flows, or test logic drift), SentinelQA parses the test script with **Babel AST (`@babel/parser`)**, traverses to the exact locator call node, replaces only the selector argument (e.g., `page.locator('#submit-btn')` → `page.locator('button:has-text("Place Order")')`), and regenerates clean code using `@babel/generator`.

5. BONUS GOALS & HACKATHON DIFFERENTIATORS (PAGE 21)

• JIRA TWO-WAY SYNC

Post execution results, failure traces, execution screenshots, and healed locator diffs directly back to the Jira ticket via Jira REST API as comments, updating status to *"QA Verified"*.

• FLAKY TEST DETECTION (MONTE CARLO)

Execute tests repeatedly (N = 5) under synthetic CPU throttling and simulated network jitter (3G) to differentiate asynchronous race conditions from deterministic locator failures.

• VISUAL REGRESSION CHECKS

Integrate Pixelmatch to compare baseline screenshots with post-change screenshots, isolating cosmetic visual modifications from functional DOM breakages.

• INTEGRATED DEMO SANDBOX APP

Include a built-in React demo application with a prominent "Simulate Refactor" toggle that mutates IDs, classes, and labels in real time to demonstrate live self-healing in front of judges.

6. RECOMMENDED TECHNICAL ARCHITECTURE (NODE.JS & REACT)

LAYER	TECHNOLOGY CHOICE	ARCHITECTURE ROLE & VALUE
Frontend	React 18 + Vite + Tailwind CSS + Lucide Icons	High-performance dashboard with real-time test execution timeline, Monaco editor for live test preview, and visual diff viewer for healed scripts.
Backend API	Node.js (v20+) + Express + TypeScript	Handles Jira REST API calls, coordinates LLM generation, launches Playwright test processes, and performs AST script patching.
Real-Time Comms	WebSockets (`ws`)	Full-duplex streaming of Playwright console output, live step progress, execution screenshots, and healing notifications.
Test Automation	Playwright (`@playwright/test`)	Headless Chromium browser execution, locator tracing, accessibility tree snapshots, and failure screenshot capture.
AI Engine	Gemini API (`@google/generative-ai`)	Rapid-latency LLM inference for initial test generation from criteria and semantic DOM element matching during self-healing.
AST Manipulation	Babel (`@babel/parser`, `@babel/traverse`, `@babel/generator`)	Parses test scripts into syntax trees, replaces broken locator strings safely, and produces deterministic code patches.
Demo Target App	Embedded React Storefront with Mutation Engine	Self-contained e-commerce store with an interactive "Mutate UI" button that breaks locators on demand for judge demonstrations.

7. COMPREHENSIVE DEPENDENCY BLUEPRINT (NODE.JS & REACT)

• BACKEND (NODE.JS / EXPRESS IN `BACKEND/PACKAGE.JSON`)

PACKAGE	ROLE IN SENTINELQA
`express`, `cors`, `dotenv`	REST API, CORS for React, and `.env` key management.
`ws`	WebSocket server streaming live test logs & step status.
`playwright`, `@playwright/test`	Headless browser runner, DOM extraction, and screenshots.
`@google/generative-ai`	Gemini SDK for AC test synthesis and semantic healing.
`@babel/parser`, `@babel/traverse`, `@babel/generator`	Parses script into AST, updates locator, regenerates code.
`axios`	Jira Cloud REST API integration (fetch & post comments).
`diff`	Generates unified git-style text diff of healed locator.
`pixelmatch`, `pngjs`	Visual regression pixel diffing for screenshots.

• FRONTEND (REACT / VITE IN `FRONTEND/PACKAGE.JSON`)

PACKAGE	ROLE IN SENTINELQA
`react`, `react-dom`, `vite`	High-speed modern single page application framework.
`tailwindcss`, `postcss`, `autoprefixer`	Monochrome utility-first responsive layout & dashboard styling.
`lucide-react`	Minimalist icons for statuses (Passed, Failed, Healed, Jira).
`@monaco-editor/react`	Embedded VS Code editor to view generated Playwright tests.
`react-diff-viewer-continued`	Side-by-side or inline code diff viewer for healed scripts.
`axios`	API client triggering generation, test runs, and Jira queries.

8. 36-HOUR IMPLEMENTATION ROADMAP WITH DEPENDENCY MAPPING

TIME WINDOW	PHASE & DEPENDENCIES	ACTION ITEMS & MILESTONE DELIVERABLES
Hours 00 – 06	**Phase 1: Testbed & Target App** `react`, `vite`, `tailwindcss`, `express`	• Initialize monorepo structure in `backend/` and `frontend/`. • Build the demo web app (E-Commerce store) with a "Simulate Refactor" toggle (swaps button IDs, changes classes, modifies button labels). • Set up basic Express HTTP API and health-check endpoints.
Hours 06 – 14	**Phase 2: Ticket Ingestion & Test Generation** `axios`, `@google/generative-ai`, `dotenv`	• Implement Jira REST API client + mock ticket JSON store (E-Commerce, Auth, Profile). • Engineer few-shot prompt for Gemini to transform Given-When-Then criteria into clean Playwright scripts. • Verify generated scripts pass basic TypeScript/JavaScript syntax validation.
Hours 14 – 20	**Phase 3: Execution & Failure Telemetry** `playwright`, `ws`, `@playwright/test`	• Build execution harness launching Playwright in headless mode. • Stream execution logs, step timings, and screenshots over WebSockets to React. • Intercept locator timeout exceptions; capture page DOM snapshot, accessibility tree, and failed locator.
Hours 20 – 28	**Phase 4: Self-Healing Logic & AST Patching** `@babel/parser`, `@babel/traverse`, `@babel/generator`, `diff`	• Implement multi-attribute candidate scoring engine comparing failed element with live DOM. • Build Babel AST rewriter to substitute broken locator string in test file without touching test logic. • Trigger automated re-run of healed script; verify green status and log unified `diff`.
Hours 28 – 33	**Phase 5: Frontend Dashboard & Live Demo Flow** `@monaco-editor/react`, `react-diff-viewer-continued`, `lucide-react`	• Complete React dashboard: Jira ticket selector, execution runner console, and Monaco code diff viewer. • Wire up Jira webhook / comment pushback showing test results and repair logs. • Conduct live test runs validating end-to-end self-healing cycle under 2 seconds.
Hours 33 – 36	**Phase 6: Rehearsal & Hardening** *End-to-End Testing*	• Rehearse 4-step live pitch (Criteria → Baseline Pass → Mutate UI → Autonomous Heal → Green Pass). • Add fallback offline mock responses to guarantee seamless presentation even if network stutters.

9. HACKATHON RULES & STRICT COMPLIANCE CHECKLIST

[x] Safe Targets Only: Execute exclusively against our local demo app or open testing sites (e.g., SauceDemo). Zero probing of external commercial apps.

[x] Transparent Audit Logging: Every repaired locator logs exact timestamps, previous selector, healed selector, semantic confidence score, and rationale.

[x] Live System Execution: All test runs, failure interceptions, and DOM inspections execute live in real browsers with zero mock delays or hardcoded values.

[x] Secure Credential Protection: Strict environment isolation via `.env` for API tokens; zero plain-text secrets in repository files or client bundles.

Summary Recommendation: SentinelQA built on Node.js and React provides maximum developer velocity and flawless integration with Playwright. Using Babel AST for in-place locator substitution ensures safe, deterministic healing without code hallucinations, creating an unbeatable hackathon demo.